Lab 01 — Foundations: Evaluation Methodology

Fundamentos de Aprendizagem Automática

Mário Antunes

2026

Overview

Class 01 · Topics 1 and 2. Lecture: `slides/01_foundations.pdf`. Demo notebooks in `notebooks/01-foundations/`: `01_learning_as_compression.ipynb`, `02_evaluation_validation.ipynb` (D4a to D4d are shown live; D4e to D4g are read in this lab), `03_pitfalls.ipynb` (leakage, D7) and `04_taxonomies.ipynb`.

Notebook for this lab: `notebooks/01-foundations/lab01_foundations.ipynb`, with helpers in `lab01_utils.py` (same folder, do not edit). Cells marked **TODO** contain `raise NotImplementedError`: replace it with your code. Every other cell is ready, and it draws the plots and prints the comparisons once your functions work.

Solution: `solutions/lab01_foundations_solution.ipynb` is the fully solved notebook. Before each function it explains how the solution was reached, and after each experiment it answers this guide's questions with the numbers the run produces. Try every task yourself first.

The rule of this lab. An evaluation is a procedure, and a procedure can be wrong without any error message: the code runs, the number looks good, and the number is false. You write the procedures (the split, the folds, the metrics, the selection inside the folds) with NumPy and run them on one problem, the breast-cancer data, and compare each with its scikit-learn or SciPy twin **on the same data and the same folds**. The notebook prints the difference; explain it.

By the end of the lab you should be able to:

1. Split data into stratified training and test parts with a seed, and judge a

model against the majority-class baseline.
2. Run k-fold cross-validation and report a mean, a standard error and a 95% confidence interval, and say how much one number depends on the split.
3. Compute a confusion matrix, precision, recall and F_1, and say when accuracy, precision, recall or F_1 misleads.
4. Show that a preprocessing step fitted outside the folds leaks the test labels, and fix it with a `Pipeline`.
5. Compare two models with the corrected resampled t-test and with McNemar's test, and explain why the naive t-test is wrong.

Part	Task	twin	Time
A	Setup: the data with polars, the classes, the baseline	—	5 min
B	Evaluation methodology, hold-out: stratified split, accuracy	`train_test_split`, `DummyClassifier`	10 min
C	Evaluation methodology, cross-validation: folds, mean, standard error, 95% interval	`cross_val_score`, `scipy.stats.t.interval`	15 min
D	Evaluation methodology, metrics: confusion matrix, precision, recall, F_1	`confusion_matrix`, `precision_recall_fscore_support`	10 min
E	Preprocessing, leakage: selection inside and outside the folds	`Pipeline`, `SelectKBest`	10 min
F	Evaluation methodology, comparing two models: the corrected t-test and McNemar (given)	`scipy.stats.binomtest`, `ttest_1samp`	10 min
G	Challenge: a permutation test; heart disease	`scipy.stats.permutation_test`	home

After Part G, a reading appendix derives the two tests of Part F (read at home, not timed).

Setup

```
cd faa
source venv/bin/activate        # README, section 6
jupyter lab notebooks/01-foundations/lab01_foundations.ipynb
```

The data are in `datasets/` (read with polars, see `datasets/README.md`): **breast cancer** (569 tumours, 30 measurements of the cell nuclei) and **heart disease** (303 patients, Part G). **The positive class is `y = 1`**, the event to detect: a malignant tumour, a disease. The breast-cancer file codes benign as 1, so `load_breast_cancer` recodes it; the demo `02_evaluation_validation.ipynb` and the slides use the same coding. Given in `lab01_utils.py`: the loaders, `make_tree`, `make_logreg` (standardization and logistic regression in a `Pipeline`), `make_noise` (data with no signal), `t_quantile`, `majority_predict`, `fold_scores`, `select_top_k`, the two tests of Part F and the plots. Every random choice takes a seed: `np.random.default_rng(seed)`.

The tree of the lab has `max_depth=3`, the tree of the demo is unpruned. The demo shows memorization, and an unpruned tree memorizes (training accuracy 1.000). The lab evaluates a model whose training and test scores stay close and whose runs are fast, so the numbers of the lab and of the demo differ on purpose.

Part A — Setup

The notebook loads the data with polars, prints the class counts and the mean of three features by class, and draws their histograms. Nothing to implement.

1. How many examples and features are there, and what share of the tumours is malignant?
2. What accuracy does a model that always answers “benign” reach? What must any model beat?

Part B — Evaluation methodology: hold-out

A model is judged on examples it did not see. The simplest protocol keeps n_{test} examples aside and fits on the other n_{train}. The split must be **random but reproducible** (a seed) and **stratified** (both parts keep the class proportions). A model must also beat the **baseline** that ignores the features.

B1. Split and accuracy

Implement `train_test_split(y, test_frac, seed)` (seed a generator with `default_rng(seed)`, take `round(test_frac * n_c)` random examples of each class for the test part; return two sorted, disjoint index arrays) and `accuracy(y_true, y_pred)`. The baseline `majority_predict` is given. The driver compares it with `DummyClassifier` and your split with scikit-learn's stratified split, and then repeats the hold-out of the tree and of the logistic regression with 30 seeds.

1. What accuracy does the baseline reach on the test part? Do your split and scikit-learn's contain the same test examples, and why do the accuracies of the same model on the two splits differ?
2. How large is the spread of the test accuracy over 30 seeds? What would be wrong with reporting the best seed?

Part C — Evaluation methodology: cross-validation

One split gives one number, and Part B showed how much it moves. **k-fold cross-validation** holds each of k folds out once, fits on the other $k-1$ and scores on the held-out one: k scores $s_1, \dots, s_k$, every example tested once. The scores are summarized by

$$\bar{s} = \frac{1}{k}\sum_{i=1}^{k} s_i, \qquad s = \sqrt{\frac{1}{k-1}\sum_{i=1}^{k}(s_i - \bar{s})^2},$$

$$\mathrm{SE} = \frac{s}{\sqrt{k}}, \qquad \mathrm{CI} = \bar{s} \pm t_{k-1}\,\mathrm{SE},$$

where t_{k-1} is the 0.975 quantile of Student's t distribution with $k-1$ degrees of freedom (2.262 for $k = 10$; given as `t_quantile(k)`). The $k-1$ in s is there because $\bar{s}$ was estimated from the same scores.

C1. Folds and out-of-fold predictions

Implement `kfold_indices(y, k, seed)` (a list of `k` sorted test-index arrays, stratified: shuffle, order the shuffled indices by class with a stable `argsort`, and give fold `j` every k-th index starting at `j`) and `cv_predict(make_model, X, y, folds)` (for each fold, fit a **new** model on the other folds and predict the held-out

one; return one prediction per example). `fold_scores(y, pred, folds)`, the accuracy inside each fold, is given.

1. How large are the folds, and what is the malignant share in each? Why must a new model be created for every fold?

C2. The interval

Implement `summarize(scores, t_q)` (return `(mean, s, se, lo, hi)`, with `ddof=1`). The driver feeds *your* folds to `cross_val_score` (same scores expected), then lets `StratifiedKFold` make its own folds, and compares your interval with `scipy.stats.t.interval`.

2. What are the mean, s, SE and the interval of each model? Do the two intervals overlap? Does the hold-out of Part B lie inside the interval of the logistic regression, and why does the tree's mean change with scikit-learn's own folds?
3. How do the standard error and the width of the interval change for $k = 2, 5, 10, 20$? Which of the two changes, and why? Are the k scores independent?

Part D — Evaluation methodology: metrics

Accuracy counts all mistakes alike. The **confusion matrix** counts the four outcomes of a binary decision (positive = malignant): TP, FP, FN and TN, with $TP + FP + FN + TN = n$. From them

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN},$$

$$F_1 = \frac{2\,\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

D1. Counts and ratios

Implement `confusion_counts(y_true, y_pred)` (returns `(TP, FP, FN, TN)`) and `precision_recall_f1(tp, fp, fn, tn)` (a ratio with an empty denominator is 0). The driver evaluates "always benign", "always malignant", the tree and the logistic regression on the out-of-fold predictions of the full data

and of a **thinned** copy with 5% malignant tumours, and compares your numbers with scikit-learn.

1. What are the accuracies of "always benign" on the full and on the thinned data, and what are its precision and recall? What does the logistic regression gain in accuracy and in recall on the thinned data? Can the tree and the logistic regression be ranked with only 19 malignant tumours?
2. Find one case where precision misleads and one where recall misleads. "Always malignant" has $F_1 = 0.543$ on the full data: what does F_1 ignore?

Part E — Preprocessing: leakage

Leakage is information from the test part reaching the fit. Any step that *learns* from the data (the mean and the standard deviation of a scaler, the list of the best features) is a part of the model and must be fitted on the training part of each fold only: inside the folds, in a `Pipeline`. The sharpest test is **pure noise**: the data of `make_noise` are Gaussian features and a coin-flip label, so an honest evaluation reports 50%.

E1. Select the features outside and inside the folds

`select_top_k(X, y, k)` (the indices of the `k` columns with the largest absolute Pearson correlation with `y`) is given. Implement `cv_selected(make_model, X, y, folds, n_keep, inside)` (the cross-validated accuracy on the `n_keep` selected features; with `inside=False` the features are selected once on all the data, with `inside=True` again in every fold on its training part). The driver uses 100 examples and 1000 noise features, compares with `Pipeline(SelectKBest(f_classif), ...)` and averages over 10 noise data sets.

1. What accuracy do the two protocols report for 1, 5, 10, 20 and 50 selected features? Which is right, and why does the other grow with the number of features?

E2. Scaling before the split

The notebook standardizes all the data before the folds, and inside each fold, for $n = 40, 100, 569$ (20 seeds each).

2. How large is the gap, and why is it so much smaller than in E1? State the rule for every step that learns from the data.

Part F — Evaluation methodology: comparing two models

Is the tree (A) really worse than the logistic regression (B)? Two tests answer it. Both are **given** in `lab01_utils.py`: you run them and interpret them (the reading appendix derives them).

Corrected resampled t-test (Nadeau and Bengio, 2003), on J paired scores ($J = 50$ for 5 x 10-fold CV; $k = 10$ stays the number of folds), with $\delta_j = s_j^A - s_j^B$:

$$T = \frac{\bar{\delta}}{\sqrt{\left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) s_\delta^2}}, \qquad p_{\text{val}} = 2\,P(t_{J-1} > |T|).$$

McNemar's test, on the same examples: n_{10} counts the examples only A gets right, n_{01} those only B gets right, $N = n_{10} + n_{01}$. McNemar (1947) gives $\chi^2 = (n_{10} - n_{01})^2/N$; Edwards (1948) subtracts 1 from $|n_{10} - n_{01}|$ before squaring. Both are approximations for $N \geq 25$: **for $N < 25$ use the exact binomial test.**

F1. Run the two tests

The driver runs 5 repeats of 10-fold cross-validation with your functions ($J = 50$ scores) and calls `corrected_t` and `naive_ttest`. It then counts $n_{11}, n_{10}, n_{01}, n_{00}$ on the out-of-fold predictions of the first repeat and calls `mcnemar_exact` and `mcnemar_chi2`, checked against `scipy.stats.binomtest`.

1. What are $\bar{\delta}$, s_δ, the naive and the corrected T and p_{val}? By what factor does the correction change T?
2. What are $n_{11}, n_{10}, n_{01}, n_{00}$ and the three p-values of McNemar (exact, McNemar 1947, Edwards 1948)? Which one do you report when $N < 25$?

F2. Two models that are equally good

The notebook repeats the comparison 200 times on pure noise (a coin-flip label), where no model is better, and counts how often each test rejects at $\alpha = 0.05$.

3. How often does each test reject? Which assumption of the naive test breaks, and what does the correction assume?

Part G — Challenge: a permutation test and another data set

G1. A permutation test

If the models were equally good, the sign of each paired difference would be a coin flip. Implement `permutation_test(diffs, n_perm, seed)`: flip the sign of every difference at random, `n_perm` times, and return $p_{\text{val}} = (1 + \#\{|\bar{\delta}^*| \geq |\bar{\delta}|\})/(n_{\text{perm}}+1)$, where $\bar{\delta}^*$ is the mean of one flipped sample. The driver applies it to the 50 score differences of Part F and to the 569 per-example differences `(pa == y) - (pb == y)` and compares with `scipy.stats.permutation_test` and with McNemar's exact value.

1. Why can the result not be smaller than $1/(n_{\text{perm}} + 1)$? How does the test on the per-example differences relate to McNemar's test? What weakness has the test on the 50 score differences?

G2. The same protocol on heart disease

Implement `compare_learners(X, y, make_a, make_b, k, n_rep, seed)`: the cross-validated accuracy and interval of each model, the corrected t-test on the $J = k \cdot n_{\text{rep}}$ scores of `n_rep` repeats of k-fold cross-validation, McNemar's exact value and the permutation value, all from your functions. The driver runs it on breast cancer and on heart disease.

2. What do the intervals, the corrected t-test, McNemar's test and the permutation test say on each data set? What would you report for heart disease, and what would you do to settle it?

Going further. Repeat Part F with the whole protocol on `german_credit` (unequal costs: a bad credit accepted costs five times a good one refused) and choose the metric that matches the costs. Or compare the corrected t-test on 20 random hold-outs of 25% with the one on 5 x 10-fold CV.

Reading at home — the two tests derived

Not a Part and not timed: read it after the lab. It derives the two tests of Part F, with the numbers of the run of Part F (the tree A against the logistic regression B, 5 x 10-fold CV).

Notation: comparing two models

Symbol	Meaning	Shape	Example
J	number of scores of a repeated hold-out or repeated k-fold CV	integer	$J = 50$
δ_j	score of model A minus score of model B on score j	scalar	$\delta_1 = -0.0175$
$\bar{\delta}$	mean of the δ_j	scalar	-0.0538
s_δ^2	variance of the δ_j, with $J-1$	scalar	0.0373^2
σ^2	true variance of one δ_j	scalar	unknown
r	correlation of two δ_j	scalar	$r \approx 0.1$
n_{train}	size of the training part of one fold or split	integer	512
n_{test}	size of the test part, $n = n_{\text{train}} + n_{\text{test}}$	integer	57
n	number of examples of the data set	integer	569
V	variance of the mean $\bar{\delta}$	scalar	unknown
S	average of s_δ^2 over many repetitions of the experiment	scalar	unknown
T	test statistic	scalar	-3.98
p_{val}	p-value	scalar	$2.3 \cdot 10^{-4}$
n_{11}	both models right	integer	522
n_{10}	only A right	integer	5
n_{01}	only B right	integer	34
n_{00}	both wrong	integer	8
N	number of disagreements, $n_{10} + n_{01}$	integer	39
i	summation index (number of heads)	integer	$0, \dots, 5$
χ^2	McNemar's statistic	scalar	21.6

The corrected resampled t-test

Question. The J scores share examples, so the δ_j are *not* independent: the ordinary t-test, which divides s_δ by $\sqrt{J}$, treats them as if they were, and finds

differences that are not there. *Idea.* Keep $\bar{\delta}$ and replace its variance estimate by one that knows about the correlation r.

Step 1: variance of the mean. *Given* J differences with variance σ^2 each and correlation r for every pair. *Goal* $\mathrm{Var}(\bar{\delta})$. The variance of a sum is the sum of the J variances and of the $J(J-1)$ ordered covariances $r\sigma^2$.

$$\begin{aligned} V &= \frac{J\sigma^2 + J\cdot(J-1)\cdot r\sigma^2}{J^2} && \text{(by the variance of a sum)} \\ &= \frac{\sigma^2}{J}\cdot\left(1+(J-1)\,r\right) && \text{(simplify)} \end{aligned}$$

Result: $V = \frac{\sigma^2}{J}(1+(J-1)r)$. With $r=0$ it is the textbook σ^2/J; any $r>0$ makes it larger.

Step 2: what the sample variance estimates. The sample variance s_δ^2 is the thing we can compute; its average is *not* σ^2. *Given* $S = \frac{1}{J-1}\left(J\sigma^2 - J\,V\right)$ (expand $\sum_j(\delta_j-\bar{\delta})^2 = \sum_j \delta_j^2 - J\bar{\delta}^2$ and take expectations; the mean μ cancels). *Goal* S in terms of σ^2 and r.

$$\begin{aligned} S &= \frac{J\sigma^2 - J\,V}{J-1} && \text{(definition of } S\text{)} \\ &= \frac{J\sigma^2 - \sigma^2\left(1+(J-1)\,r\right)}{J-1} && \text{(substitute } V \text{ from step 1)} \\ &= \frac{\sigma^2\,(J-1)\,(1-r)}{J-1} && \text{(factor)} \\ &= \sigma^2\,(1-r) && \text{(simplify)} \end{aligned}$$

Result: $S = \sigma^2(1-r)$, so s_δ^2 estimates $\sigma^2(1-r)$, *smaller* than σ^2: correlated scores look more alike than independent ones.

Step 3: the corrected variance. *So far:* $S = \sigma^2(1-r)$, hence $\sigma^2 = S/(1-r)$, and we estimate S by s_δ^2. *Goal:* an estimate of V from s_δ^2 alone.

$$\begin{aligned} V &= \frac{\sigma^2}{J}\cdot\left(1+(J-1)\,r\right) && \text{(definition of } V\text{)} \\ &= \frac{s_\delta^2}{J\cdot(1-r)}\cdot\left(1+(J-1)\,r\right) && \text{(substitute } \sigma^2 \text{ from step 2)} \\ &= s_\delta^2\cdot\left(\frac{1}{J}+\frac{r}{1-r}\right) && \text{(split the fraction)} \end{aligned}$$

Result: $\widehat{V} = s_\delta^2\big(\frac{1}{J} + \frac{r}{1-r}\big)$. The naive test is the case $r = 0$.

Step 4: the correlation. Nadeau and Bengio *assume* that two random test parts of n_{test} examples out of n overlap in a share n_{test}/n of their examples, and that this share is the correlation: $r \approx n_{\text{test}}/n$. It is a heuristic, not a theorem. *Given* $r = n_{\text{test}}/n$ and $n = n_{\text{train}} + n_{\text{test}}$.

$$\begin{aligned} \frac{r}{1-r} &= \frac{n_{\text{test}}/n}{1 - n_{\text{test}}/n} && \text{(assumption } r = n_{\text{test}}/n\text{)} \\ &= \frac{n_{\text{test}}}{n - n_{\text{test}}} && \text{(simplify)} \\ &= \frac{n_{\text{test}}}{n_{\text{train}}} && \text{(substitute } n = n_{\text{train}} + n_{\text{test}}\text{)} \end{aligned}$$

Result. The statistic compares $\bar{\delta}$ with the square root of $\widehat{V}$ and is referred to a Student t distribution with $J - 1$ degrees of freedom:

$$T = \frac{\bar{\delta}}{\sqrt{\left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) s_\delta^2}}, \qquad p_{\text{val}} = 2\,P\big(t_{J-1} > |T|\big).$$

Worked numbers (tree against logistic regression, $J = 50$, $n_{\text{test}}/n_{\text{train}} = 1/9$, $\bar{\delta} = -0.0538$, $s_\delta = 0.0373$):

$$\begin{aligned} T &= \frac{-0.0538}{\sqrt{\left(\frac{1}{50} + \frac{1}{9}\right) \cdot 0.0373^2}} && \text{(definition of } T\text{)} \\ &= \frac{-0.0538}{\sqrt{\frac{59}{450} \cdot 0.00139129}} && \text{(evaluate)} \end{aligned}$$

This is $T = -3.98$ (the notebook prints -3.980 from the unrounded values). The naive statistic uses $1/50 = 0.02$ in place of 0.1311 and is $\sqrt{0.1311/0.02} = 2.56$ times larger (-10.2). The notebook prints both and checks this ratio.

McNemar's test

Question. Two models predict the same 569 examples. Where they agree, nothing separates them; only the **disagreements** carry information. *Idea.* If the models were equally good, a disagreement would favour A or B with the same probability,

so n_{10} is the number of heads in $N = n_{10} + n_{01}$ fair tosses.

Given $n_{10} \sim$ Binomial$(N, \frac{1}{2})$ under the null hypothesis, with mean $N/2$ and variance $N/4$. The **exact** two-sided p-value is the probability of a split of the disagreements at least as lopsided as the observed one. With $m = \min(n_{10}, n_{01})$ the two tails are equal by symmetry:

$$p_{\text{val}} = 2\,P(\text{Binomial}(N, \tfrac{1}{2}) \le m) = \frac{2}{2^N} \sum_{i=0}^{m} \binom{N}{i}, \qquad \text{capped at } 1.$$

Worked numbers ($n_{10} = 5, n_{01} = 34, N = 39, m = 5$):

$$\begin{aligned} p_{\text{val}} &= \frac{2}{2^{39}} \sum_{i=0}^{5} \binom{39}{i} && \text{(definition of the exact } p\text{-value)} \\ &= \frac{2 \cdot 667928}{549755813888} && \text{(evaluate)} \\ &= 2.42987 \cdot 10^{-6} && \text{(evaluate)} \end{aligned}$$

The normal approximation of the binomial gives the **statistic**: standardize n_{10} and square it, so that $\chi^2 = z^2$ is referred to a chi-squared distribution with one degree of freedom.

$$\begin{aligned} \chi^2 &= \left(\frac{n_{10} - N/2}{\sqrt{N/4}} \right)^2 && \text{(definition of } z^2\text{)} \\ &= \frac{4\,(n_{10} - N/2)^2}{N} && \text{(simplify)} \\ &= \frac{4\,\big((n_{10} - n_{01})/2\big)^2}{N} && \text{(substitute } N = n_{10} + n_{01}\text{)} \\ &= \frac{(n_{10} - n_{01})^2}{N} && \text{(simplify)} \end{aligned}$$

Result: $\chi^2 = (n_{10} - n_{01})^2 / N$ is **McNemar's (1947) statistic**; here $29^2/39 = 21.6$ and $p_{\text{val}} = 3.4 \cdot 10^{-6}$. The approximation is smooth where the exact distribution has steps, and it is poor when N is small (the usual rule is $N \ge 25$; **below it use the exact value**). **Edwards (1948)** proposed a continuity correction: subtract 1 from $|n_{10} - n_{01}|$ before squaring, which gives $28^2/39 = 20.1$ and $p_{\text{val}} = 7.3 \cdot 10^{-6}$ here. When in doubt use the exact value.